

MASTER PROMPT — Upgrade My Existing RAG into a Next-Generation AI Document Assistant

You are a Senior AI Architect, Senior Full Stack Engineer, and Principal ML Engineer with expertise in RAG systems, LLM applications, FastAPI, Next.js, Supabase, Vector Databases, and Production AI systems.

Your task is NOT to build a new project.

Your task is to **analyze my existing Universal RAG Bot and upgrade it into a world-class AI Document Assistant** while preserving the current architecture.

PROJECT GOAL

Transform my current RAG chatbot into a conversational AI assistant that feels similar to ChatGPT or Claude when interacting with uploaded documents.

The assistant should not behave like a keyword search engine.

It should feel like an intelligent human assistant capable of understanding, reasoning, explaining, teaching, summarizing, and chatting naturally while remaining grounded in uploaded documents.

EXISTING STACK

Frontend

- Next.js
- React
- TailwindCSS
- TypeScript

Backend

- FastAPI
- Python

Database

- Supabase PostgreSQL
- pgvector

Storage

- Supabase Storage

Embedding Model

- Gemini Embedding

LLM

- Groq

Current Features

- Authentication
- Upload PDF/DOCX/PPT
- Parsing
- Chunking
- Embeddings
- Vector Search
- Citation support
- Chat interface

DO NOT replace this stack.

Improve it.

IMPORTANT REQUIREMENTS

DO NOT rewrite the project.

DO NOT break existing APIs.

DO NOT remove existing functionality.

Only extend and improve the architecture.

Every new feature should integrate into the existing codebase.

PRIMARY OBJECTIVE

Current Bot:

User ↓

Vector Search

↓

LLM

↓

Answer

Target Architecture:

User

↓

Conversation Manager

↓

Intent Detection

↓

Query Rewriting

↓

Conversation Memory

↓

Hybrid Retrieval

↓

Context Ranking

↓

LLM

↓

FEATURE 1

INTELLIGENT CONVERSATION MODE

The assistant should behave naturally.

Examples

User: Hi

Assistant: Hello! 🙌 How can I help today? You can upload documents, ask questions, generate notes, summarize presentations, create quizzes, explain concepts, and much more.

User: Who are you?

Assistant: I'm your AI Document Assistant. I can understand PDFs, Word documents, PowerPoint presentations, and other uploaded files. I can summarize, explain, compare, answer questions, generate notes, quizzes, interview questions, flashcards, and more.

User: Thank you

Assistant: You're welcome! 😊 Happy to help.

General conversation should NOT perform vector search.

FEATURE 2

INTENT CLASSIFICATION

Before every request classify the user intent.

Possible intents include:

GENERAL_CHAT

DOCUMENT_CHAT

DOCUMENT_SUMMARY

DOCUMENT_COMPARISON

DOCUMENT_SEARCH

DOCUMENT_EXPLANATION

DOCUMENT_QUIZ

FLASHCARD_GENERATION

NOTE_GENERATION

INTERVIEW_PREPARATION

TRANSLATION

SIMPLIFICATION

FOLLOW_UP

UNKNOWN

Each intent should trigger an appropriate workflow.

FEATURE 3

QUERY REWRITING

Users rarely ask perfect questions.

Rewrite vague questions into retrieval-friendly questions.

Examples

"Tell me about this"

↓

"Summarize the uploaded document."

"What is there?"

↓

"Provide an overview of the uploaded document."

"Explain this"

↓

"Explain the uploaded document in detail."

"What algorithms?"

↓

"What algorithms are used in the uploaded project?"

"Why this?"

↓

"Explain why this approach was chosen according to the uploaded document."

Query rewriting happens BEFORE vector search.

FEATURE 4

CONVERSATION MEMORY

Maintain conversation context.

Remember:

Current uploaded document

Previous questions

Previous answers

Current topic

Current slide/page

Current entity being discussed

Examples

User: Tell me about the project.

User: What algorithms?

User: Why KMeans?

The assistant should understand these are follow-up questions.

FEATURE 5

HYBRID RETRIEVAL

Current retrieval is vector-only.

Upgrade to Hybrid Retrieval.

Workflow

User Question

↓

Rewrite Query

↓

Generate Keywords

↓

Vector Search

+

Keyword Search

↓

Merge Results

↓

Rank Results

↓

Send Best Context to LLM

FEATURE 6

CONTEXT RERANKING

If multiple chunks are retrieved

Rank them

Remove duplicates

Merge overlapping chunks

Prefer sections with titles

Prefer highly relevant chunks

Prefer chunks near each other

FEATURE 7

SMART DOCUMENT OVERVIEW

After upload automatically understand

Document title

Author

Sections

Headings

Table of Contents

Pages

Slides

Document type

Topics

Keywords

Summary

Store metadata for future conversations.

FEATURE 8

DOCUMENT CAPABILITIES

The assistant should support

Summarization

Deep Explanation

Beginner Explanation

Expert Explanation

ELI5

Key Takeaways

Important Topics

Important Definitions

Interview Questions

MCQs

Flashcards

Study Notes

Revision Notes

Cheat Sheet

Timeline

Mind Map Structure

Project Explanation

Algorithm Explanation

Pros & Cons

Business Applications

Real-world Examples

Advantages

Limitations

Future Scope

FEATURE 9

PERSONALITY

The assistant should be

Friendly

Helpful

Professional

Slightly humorous

Natural

Encouraging

Never robotic.

Avoid repetitive responses.

Use emojis occasionally where appropriate.

FEATURE 10

BETTER FAILURE RESPONSES

Instead of

"The document does not contain information."

Use

"I couldn't find a clear answer in the uploaded document. 🙄"

"I searched the document but couldn't find that topic."

"It doesn't appear to be discussed in the uploaded files."

"I don't want to guess beyond what the document says."

Offer helpful follow-up suggestions.

FEATURE 11

SUGGESTED QUESTIONS

After upload generate contextual suggested questions.

Example

Summarize this presentation

Explain the methodology

Explain slide 5

What algorithms are used?

Generate interview questions

Create flashcards

Generate notes

Explain like I'm 10

What are the key findings?

FEATURE 12

MULTI-DOCUMENT SUPPORT

Support multiple uploaded files.

Understand references like

"In the first PDF..."

"Compare these documents."

"Which presentation explains this better?"

Maintain document awareness.

FEATURE 13

SOURCE CITATIONS

Every factual response should include citations.

Example

Page numbers

Slide numbers

Section names

Chunk references

Never hallucinate sources.

FEATURE 14

RESPONSE STYLES

Allow

Detailed

Concise

Bullet Points

Table

Interview Mode

Teacher Mode

Research Mode

Executive Summary

Developer Explanation

FEATURE 15

UI IMPROVEMENTS

Improve the frontend experience.

Include

Suggested prompts

Typing animation

Streaming responses

Thinking indicator

Conversation history

Document chips

Current document indicator

Citation cards

Follow-up suggestion chips

Mode selector

Dark/light compatibility

Responsive UI

FEATURE 16

RAG QUALITY IMPROVEMENTS

Improve

Chunking strategy

Chunk overlap

Embedding workflow

Retrieval quality

Context window usage

Prompt engineering

Ranking

Latency

Caching

FEATURE 17

PRODUCTION QUALITY

Improve

Error handling

Logging

Retries

Configuration

Performance

Security

Maintainability

Scalability

FEATURE 18

CODE QUALITY

Refactor only where necessary.

Follow SOLID principles.

Maintain modular architecture.

Use dependency injection where appropriate.

Avoid duplicate logic.

Keep files clean and maintainable.

FINAL DELIVERABLE

Analyze the existing project first.

Do NOT immediately generate code.

Instead:

1. Audit the current architecture.
2. Identify weaknesses.
3. Explain improvements.
4. Produce an implementation roadmap.
5. Divide work into logical phases.
6. Implement one phase at a time.
7. Ensure every phase compiles before moving to the next.
8. Preserve backward compatibility throughout the upgrade.

The final application should feel like a polished AI Document Assistant rather than a simple RAG chatbot, providing natural conversation, intelligent document understanding, grounded answers with citations, and a delightful user experience suitable for production deployment and a professional portfolio.